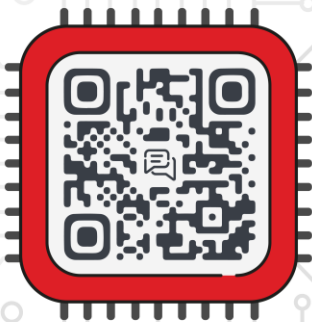




9

13

1950 sp

نظري

A simplified MIPS implementation

د.خولة العلي



محتوى مجاني غير مخصص للبيع التجاري

بنيان الحاسوب ا

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ

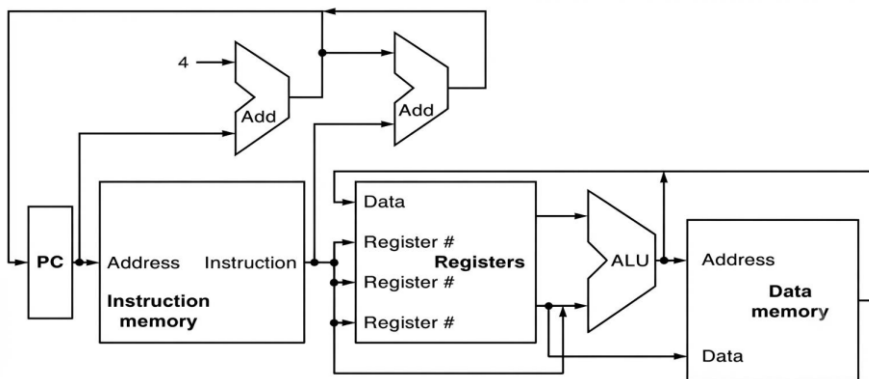
سنتعرف عن كيفية بناء هذه الدارات التي تقوم بتنفيذ التعليمات التي يقوم بها المعالج

Building a MIPS implementation

تقسم إلى ثلاثة أنواع ، أي بناء Alu تقوم بتنفيذ التعليمات

اختيار استراتيجيات التنفيذ تؤثر على الـ cpi rate

عدد دورات الساعة للتعليم الواحدة



هذه الرسوم تعطينا النظرة العامة لمكونات الحاسب الأساسية

ملاحظة: الـ Alu و (Control Unit) CU عبارة عن دارات إلكترونية .

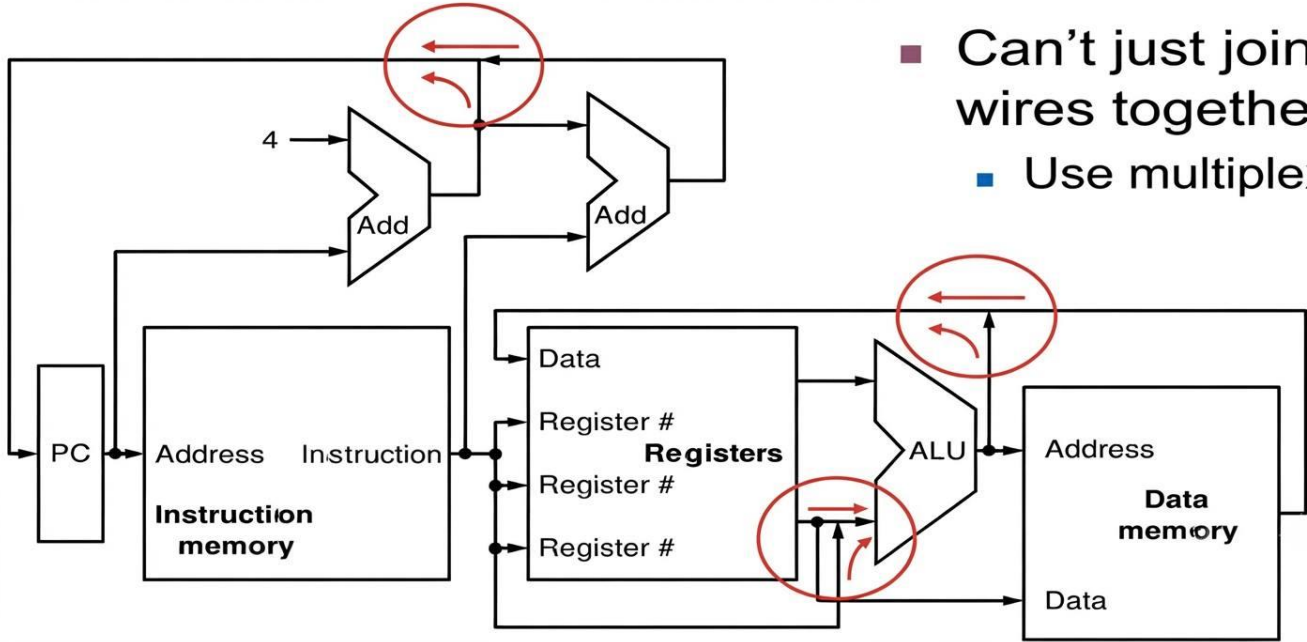
وتركز على مسار البيانات (Data path) بدون وحدة التحكم عن بعد تحتوي على :

1. Pc(program counter) : عنوان التعليمات الحالية
2. Instruction Memory : تخزين التعليمات ، نقرأ منها باستخدام العنوان pc
3. Register 1 and 2 : قراءة سجلين من مجموعة المسجلات
4. Alu : تنفذ التعليمات الحسابية والمنطقية
5. Data Memory : ذاكرة البيانات للتحميل والتخزين
6. Add (الجامع العلوي) : يزيد pc بمقدار 4 للتعليمات التالية

حتى نوصل وصلات الكهرباء لا يمكن وصل إلا شريط واحد

لكن عندما نريد أكثر من وصلة في مكان واحد، هذا الوصل لا يقبل إلا وصل واحد لذلك نلجأ إلى استخدام الـ selector
نربط أكثر من تعليمة وحسب ترميز كل تعليمة يتم اختيار أي منها سيتم تمريرها عبر الدارة.

- Can't just join wires together
- Use multiplexers



Instruction Execute Process

أي تعليمة يتم تنفيذها عبر عدة مراحل:

1. جلب التعليمات: إرسال قيمة عداد البرنامج إلى ذاكرة التعليمات، فيتم جلب التعليمة الموجودة في ذلك العنوان
2. فك ترميز التعليمات وقراءة السجلات
3. تنفيذ التعليمات يعتمد على القيم الفعلية

ملاحظة:

■ التعليمات ثلاث أنواع:

1. حسابية و منطقية مثل: sub-or-slt-add-and من نوع R-Type
 2. تعليمات القفز والتحكم تنقسم إلى: beq و bne من نوع I-Type و (jump) من نوع J-Type
 3. تعليمات الذاكرة (نقل البيانات من إلى الذاكرة) مثل: lw و sw من نوع I-Type
- بغض النظر عن نوع التعليمات فإن الخطوتين الأوليين من عملية تنفيذ التعليمات هما نفس الشيء

ما يجب أن تفعل وحدة ALU:

1. تعطي نتيجة حسابية، النتيجة إما تخزن في سجل أو في الذاكرة
2. حساب عنوان الذاكرة
3. ثم مقارنة الفروع

سؤال: كيف تتم عملية الوصول لبيانات الذاكرة سواء قراءة أو كتابة؟

بتنفيذ lw و sw ثم بعد تنفيذ الثلاث مراحل عندما يتم الحصول على التعليمة، يقوم الـ pc بتغيير قيمته $pc+4 \leftarrow$

لماذا 4؟

لأن العنوانه تقام بالـ byte وكل كلمة تمثل بـ 4-بايت

Digital logic Design

نحتاج لمعرفة آلية عمل العتاد الصلب منطقياً وكيف تتم عملية التزامن مع ساعة المعالج

Data path element in MIPS أفراد الدارة (المكونات التي تقوم بتنفيذ التعليمات):

1. Combinational element (تجميعية).
تعمل على قيم البيانات $\leftarrow$ الخرج يعتمد على الدخل (مثل ALU - and) لا يوجد ذاكرة داخلية
2. Sequential element (تتابعية).
عناصر الذاكرة مثل (سجل - ذاكرة) يحتوي على الأقل مدخلان أما البيانات يجب أن تكتب في عنصر ما أو إشارة الزمن تحدد متى تكتب هذه العملية.
* لديها خرج واحد (إما قراءة القيمة أو كتابة القيمة)

Clocking methodology طرق التزامن

منهجية تحدد متى يمكن قراءة البيانات ومتى يمكن كتابتها، يمكن أن تضمن أن جميع الإشارات تنتقل من عنصر تسلسلي (1) عبر المنطق التوافقي إلى عنصر تسلسلي (2) خلال دورة ساعة واحدة.
منهجية الحافة تسمح باستخدام قراءة المحتوى في السجلات وترسل القيمة عبر المنطق التوافقي (الدارة التركيبية) وكتابة القيم في السجلات في نفس دورة الساعة

Sequential elements العناصر التسلسلية

هي عبارة عن سجلات تتحكم بالكتابة.
ويتم التحديث فقط على حافة الساعة عندما يكون مدخل التحكم في الكتابة مساوياً (1) وتستخدم عندما تكون القيمة المخزنة مطلوبة لاحقاً.

لاحظ: عند الكتابة نحن بحاجة إلى قيمة البيانات التي تكتب وإلى إشارة توقيت مع وصله تتم عملية الكتابة وبالنتيجة تكون تمت عملية الكتابة.

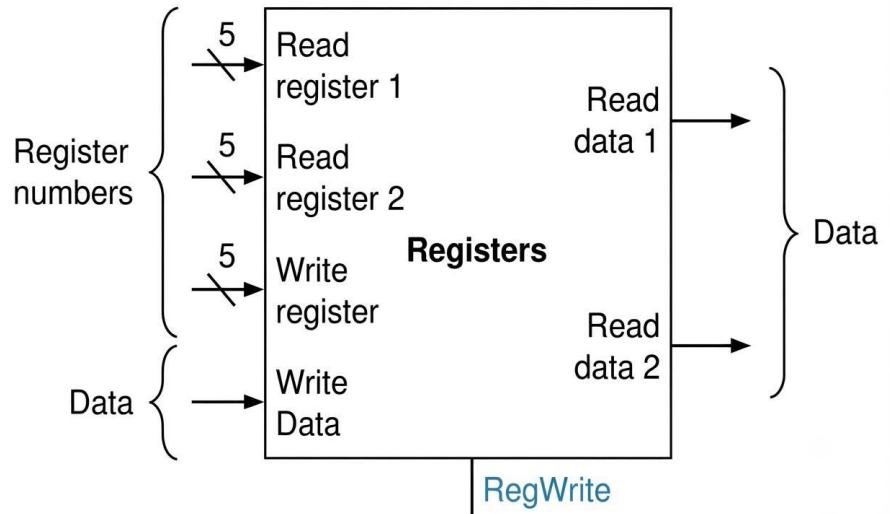
Building a Datapath

- عناصر تعالج البيانات والعناوين في المعالج cpu مثل السجلات – Mux's – memories
- سوف نبني عناصر MIPS ولكن خطوة تلو الأخرى (بشكل متزايد)

العناصر الرئيسية المطلوبة لتنفيذ كل نوع من تعليمات MIPS هي:

- 1- Instruction memory ← تحتاج إلى وحدة الذاكرة لتخزين التعليمات في البرنامج
- 2- تنفيذ بسيط ← فقط بحاجة إلى عنوان للكتابة أو القراءة
لا يوجد حاجة إلى إشارة كتابة
- Pc : مسؤول عن التنفيذ التتابعي للبرنامج
- 1- يحتوي على السجلات التي تحمل عنوان التعليمات الحالية (المراد تنفيذها)
- 2- كل تعليمات يكون حجمها 32 بت
- Adder ← بحاجته لإضافة 4 إلى كل pc لينتقل إلى التعليمات التالية (العنوان الجديد يشير إليه)
- Register file : 32 سجل يوجد في المعالج، يمكنه القراءة أو الكتابة وتكون على سجل محدد
- للكتابة : يزود باسم المسجل وعدم وجود إشارة تحكم
- للكتابة نحتاج إلى أربعة مداخل : 1- رقم السجل 2- بيانات للكتابة 3- إشارة تحكم بالكتابة 4- ساعة

كل 5 بايت هي عتاد
المصادر (الأول والثاني)
والخمسة الأخيرة هي
للوجهة وتمثل رقم
السجل (مكانه)
((أين نكتب النتيجة؟
(في سجل الوجهة))



- Alu : ال ALU (الوحدة الحسابية والمنطقية): دائرة إلكترونية تنفذ العمليات الحسابية والمنطقية (مثل add , sub , or , and) على البيانات المقروءة من السجلات Registers.
- البنية الداخلية: تبني هذه الوحدة من بوابات أساسية هي AND و OR و Inverter (NOT)، بالإضافة إلى Multiplexer يُستخدم لاختيار نوع العملية المطلوب تنفيذها من بين العمليات المتاحة.
- المدخلات: قيمتان منفصلتان (تسميان عادة A و B)، كل منهما بحجم 32 بت.
- المخرجات:

الناتج: (Result) ناتج العملية، وحجمه 32 بت.

• **الصفر (Zero):** بت واحد منفصل عن Result ، يأخذ القيمة 1 فقط إذا كانت نتيجة العملية (Result) تساوي صفراً، ويأخذ القيمة 0 في أي حالة أخرى.

• **إشارة التحكم (Control Signal):** تتكون من 4 بتات، وهي التي تُحدّد للـ ALU أي عملية من العمليات المتاحة يجب تنفيذها في كل تعليمة. كل قيمة من قيم هذه البتات الأربع تقابل عملية محددة (مثال: 0010 تعني الجمع ، 0110 تعني الطرح، 0000 تعني AND ، 0001 تعني OR).

غير مطلوب حفظ العملية المقابلة لكل قيمة من الأربع بتات

Data memory : لديها مدخلان : العنوان وبيانات الكتابة ، ومخرج واحد : نتيجة القراءة

حيث أنه يتم التحكم عن طريق MemWrite , MemRead لتحديد إذا كنت سأكتب أو أقرأ

Sign Extension unit : وحدة توسيع الإشارة (البتات)

مثال: لدينا قيمة عبارة عن 16 بت ، باقي القيم (البتات الفارغة) نضعها :

صفر $\Leftarrow$ إذا كانت القيمة + واحد $\Leftarrow$ إذا كانت القيمة -

أي أنها تعمل على توسيع الإشارة بإضافة القيم حسب الحالة (دخل 16 بت خرج 32 بت)

في مرحلة جلب التعليمات Fetch في Building a Datapath

- تعد المرحلة الأولى في تنفيذ التعليمة
- أحتاج إلى pc و خلايا الذاكرة و adder
- يقوم adder بإضافة 4 إلى القيمة السابقة وبالتالي بعد الإضافة ندخل القيمة الجديدة إلى pc
- الpc ينتقل إلى القيمة في الذاكرة $\Leftarrow$ لتحديد المكان الموجود الذي يحمل هذا العنوان وينتظر الحصول على إشارة قراءة ليتم قراءة هذه التعليمة
- أي أنه من أجل التحضير لتنفيذ التعليمة اللاحقة نقوم بزيادة pc بمقدار 4

بالنسبة للتنفيذ (Execute) Building a Datapath

كيف يتم تنفيذ التعليمات اللاحقة؟

1. **Alu** تعليمات مثل (add - sub - and - or - slt) وهي تعليمات من نوع R-Type

2. اختبار التساوي Branch equal (beq) وتعليمات القفز jump (j)

3. **Load / Store** : (lw-sw) وهي تعليمات من نوع I-Type

تنفيذ التعليمة هناك خطوتان تشتركان لجميع التعليمات وهي Fetch, Decoding ثم التنفيذ يختلف حسب التعليمة

أول عملية تنفيذ R-format : التعليمات المتطلبة للتنفيذ (add - sub - and - or - nor - slt)

تتطلب: 1- سجلان للقراءة (السجلات تحمل المصدر)

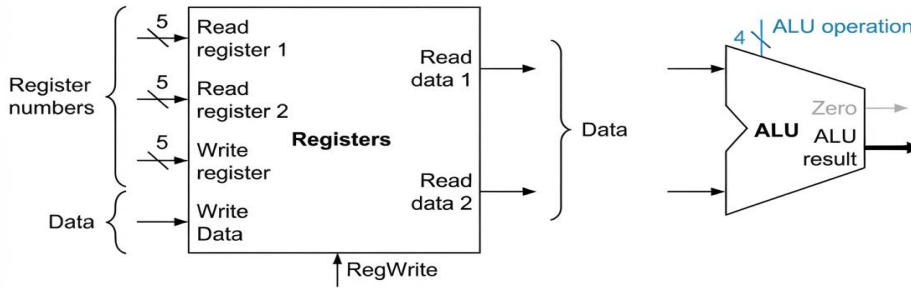
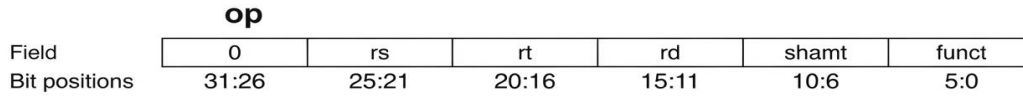
2- تنفيذ تعليمات Alu على محتوى السجل

3- كتابة النتيجة في السجل

لذلك نحتاج إلى:

1 - **Register file :** للكتابة والقراءة من السجلات

2 - **Alu :** لتنفيذ التعليمة على القيم المقروءة من الـ register



1. الذواكر هي عبارة عن Register file أو memory

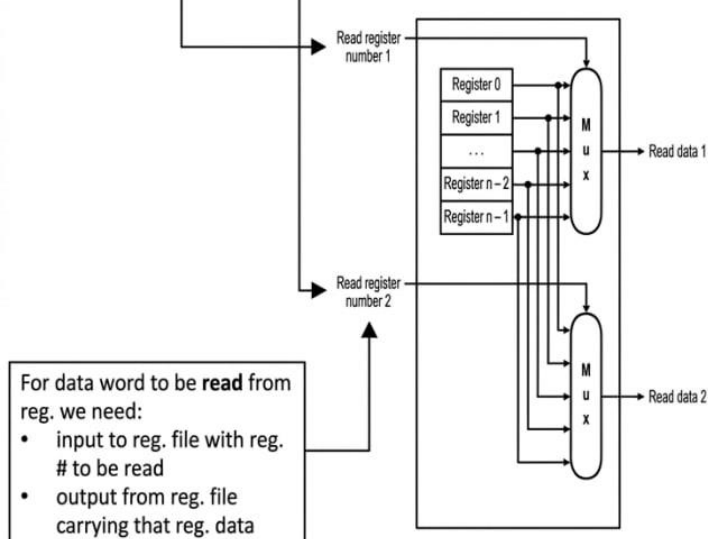
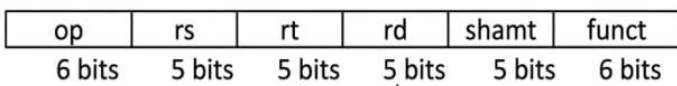
2. في الشكل السابق لدينا Register file وبما أن التعليمية من النوع R-Type فإن الدخل لكل من السجلات الثلاثة يكون بحجم 5bit, ال 5 الأولى والثانية هي عبارة عن Source Register الأول والثاني أما ال 5 الأخيرة عبارة عن Destination

:Building a Datapath (Register file)

أي لقراءة بيانات كاملة (32 بت) من سجل معين يلزم:

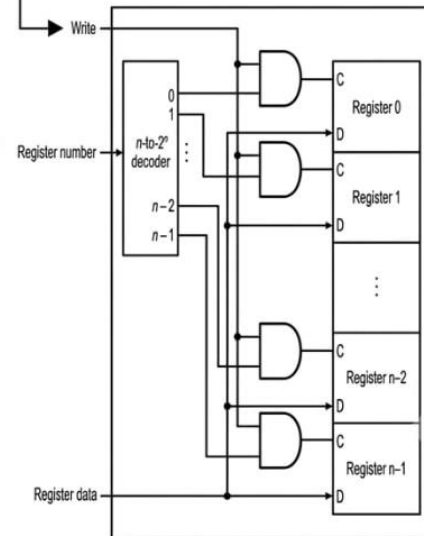
- 1- تزويد ملف السجلات برقم السجل المطلوب (كل مدخل القراءة)
 - 2- الحصول على مخرج من ملف السجلات يحتوي على قيمة ذلك السجل
- ولكتابة بيانات كاملة في ملف السجلات يلزمنا:

- 1- مدخلان: الأول: يحتوي على رقم السجل المطلوب الكتابة فيه
 - الثاني: يحتوي على القيمة المراد كتابتها
- 2- إشارة تحكم تسمى تمكين الكتابة (لتفعيل عملية الكتابة)



To write a data word to reg. file we need:

- two inputs: one with reg. # to be written and one to supply data
- Write enable



For data word to be read from reg. we need:

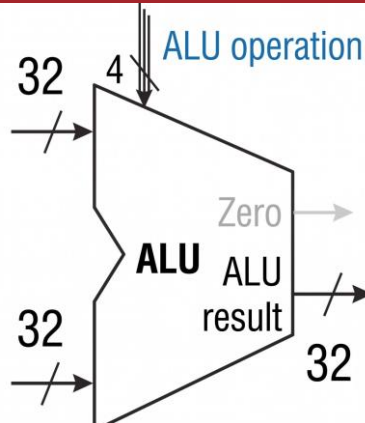
- input to reg. file with reg. # to be read
- output from reg. file carrying that reg. data

: Building a Datapath : Alu (component)

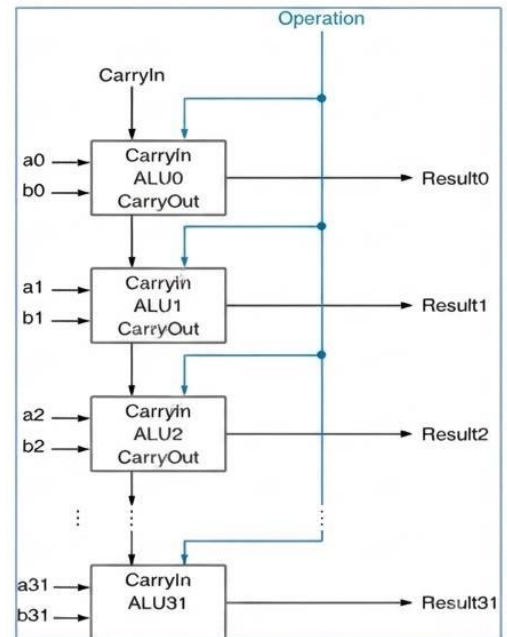
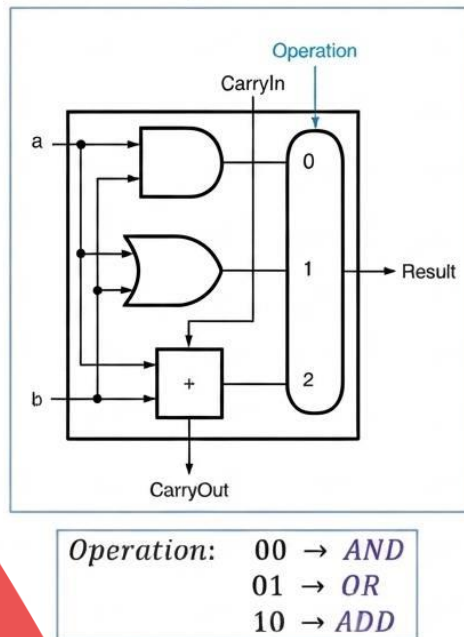
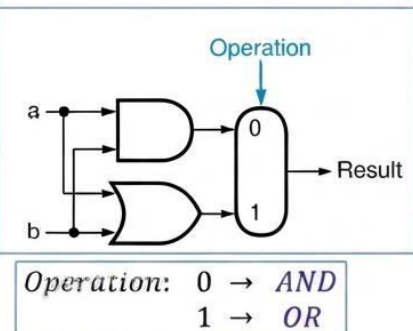
المدخلات : عملية ال ALU تتطلب 4 بت ومعاملان لكل منها (32 بت)

المخرجات : نتيجة ال ALU (32 بت) و صفر (1 بت)

نحتاج لإنجاز هذ التعليمات add - sub - and - or - nor - slt (32 بت ALU) ثم نبني منه (32 بت ALU)



Building a Datapath: ALU(AND, OR, Add)



32-bit: Repeat and connect CarryOut to CarryIn 32 ALUs

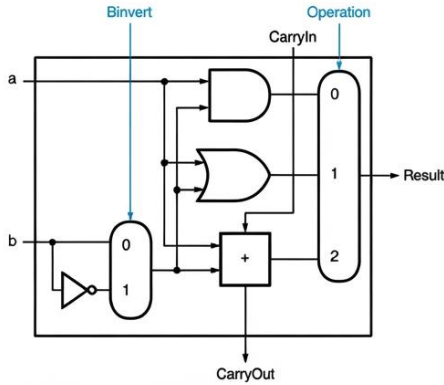


Building a Datapath: ALU(Sub, NOR)

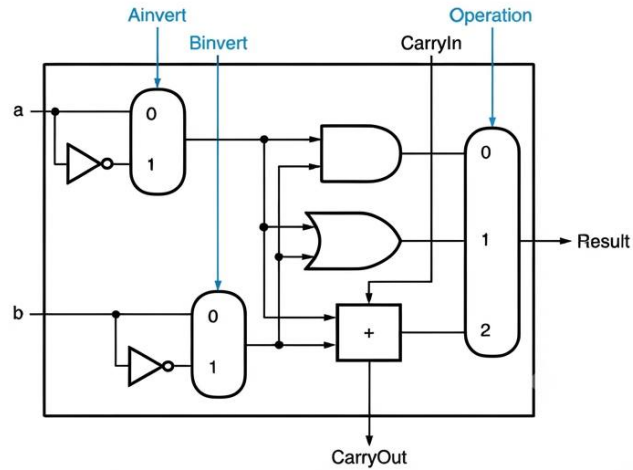
subtraction: $a - b = a + (-b) = a + \bar{b} + 1$

We need:

- 2×1 mux to choose b and $\bar{b}$
- ($\text{Binvert} = 1 \Rightarrow \text{sub}$), $\text{carryIn} = 1$



NOR operation: $(\overline{a + b}) = \bar{a} \cdot \bar{b}$



: Building a Datapath : ALU (slt)

هي عملية مقارنة : تنتج 1 إذا كان $rs < rt$

و 0 في باقي الحالات ... ولمعرفة النتيجة نقوم بالطرح $a-b$ إذا كانت النتيجة أصغر من الـ 0 تكون b

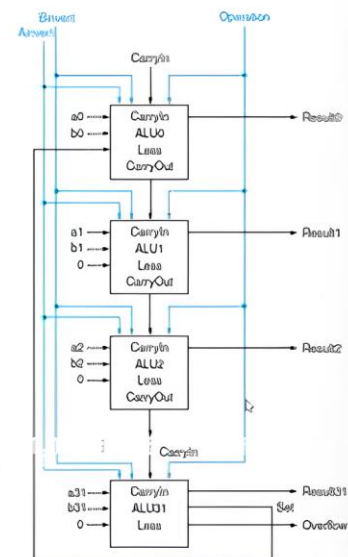
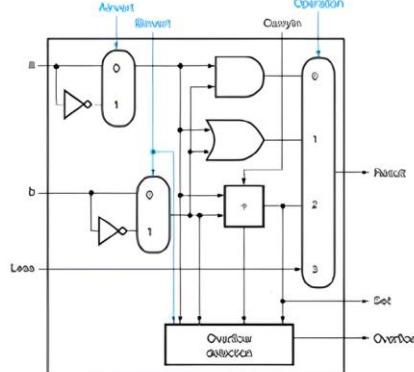
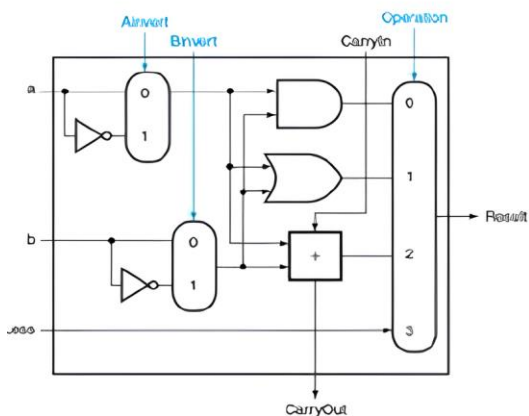
أكبر من a وبالتالي يكون البت 1

لذلك نقوم بتوصيل بت الإشارة الصادر من مخرج الجامع إلى البت الأقل أهمية (LSB) للحصول على المجموع إذا كان أصغر.

أما الـ 31 بت العلوية من مدخل Less تكون جميعها أصفاراً.

Building a Datapath: ALU(slt)

- The operation produces 1 if $rs < rt$, and 0 otherwise
- expand the three input mux to have slt result
- slt operation: $(a - b) < 0 \Rightarrow a < b$, so if sgn bit is 1, then $a < b$
- so connect the sgn bit from the adder output to the LSB bit to get set on less than (all upper 31 bits of Less input are 0's)



: Building a Datapath : Alu (Branch)

$$\text{if}(a - b = 0) \rightarrow a = b$$

لذلك إذا كانت النتيجة تساوي الصفر يمكن اختبار التساوي

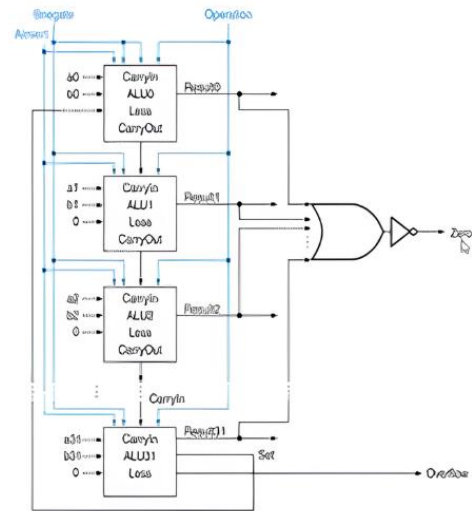
$$\text{zero} = \overline{(R31 + R30 + \dots + R0)}$$

Building a Datapath: ALU(Branch)

Branch operations:

- $\text{if}(a - b = 0) \Rightarrow a = b$
- So if the result is 0 we can test for equality

$$\text{Zero} = \overline{(\text{Result}31 + \text{Result}30 + \dots + \text{Result}0)}$$

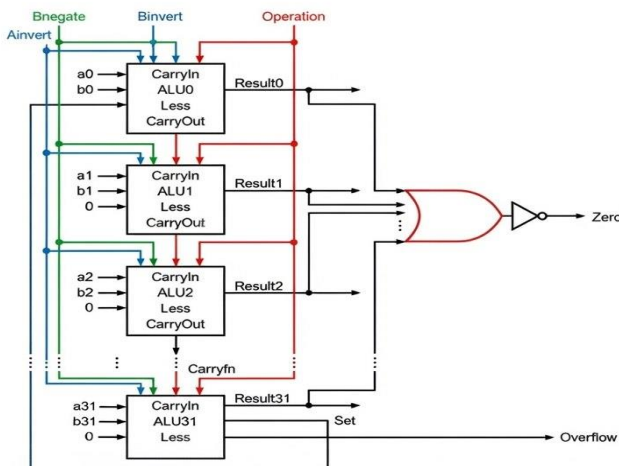


: Building a Datapath Alu (operation)

تحكم Alu

- (بت 1) Ainvert
 - (بت 1) Binvert
 - العملية (2 بت)
 - أي إنني بحاجة إلى 4 إشارات تحكم
- يمثلو الـ 4 بت الخاصة بـ Alu

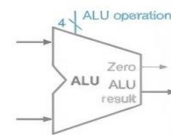
Building a Datapath: ALU (Operation)



• ALU control

- Ainvert (1 bit)
- Binvert (1 bit)
- Operation (2 bits)

4-bit control lines for the ALU



: Building a Datapath lw and sw

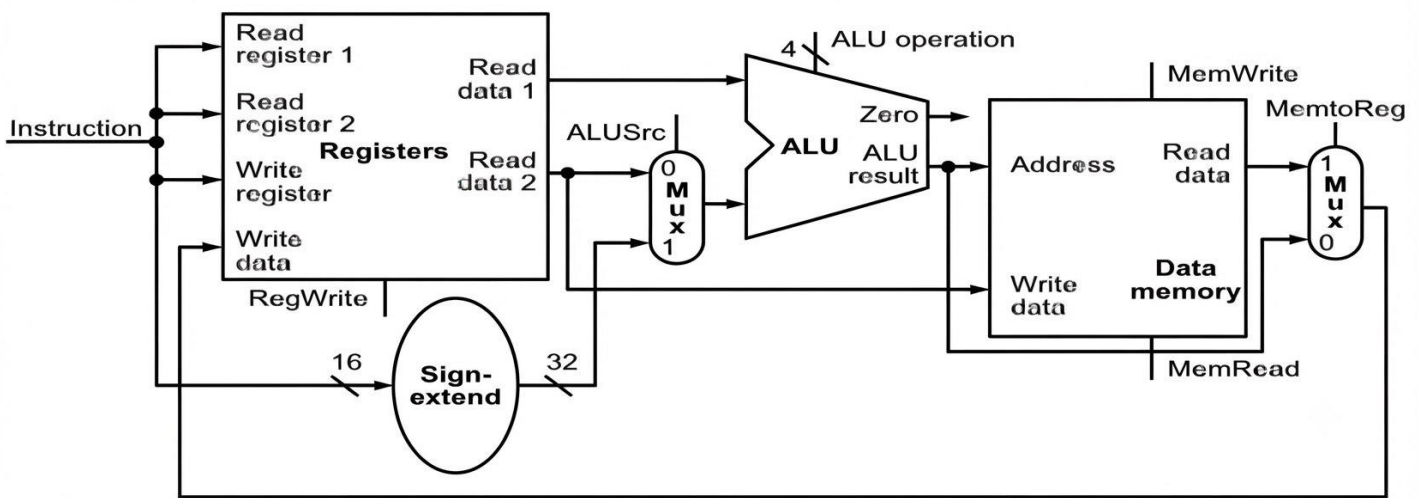
$Lw \$t1, offset-value (\$t2) \Leftarrow$ الكتابة في السجل

$sw \$t1, offset-value (\$t2) \Leftarrow$ القراءة من السجل

Offset-value

حيث offset-value هي مقدار الانزياح عن السجل القاعدي

في عملية الكتابة والقراءة lw/sw نحتاج إلى data memory وفي عملية الانزياح حسب offset-value نحتاج إلى (register /file - Alu - and sign Extend Unit)



: Building a Datapath: (beq)

Beq $\$t1, \$t2, offset$

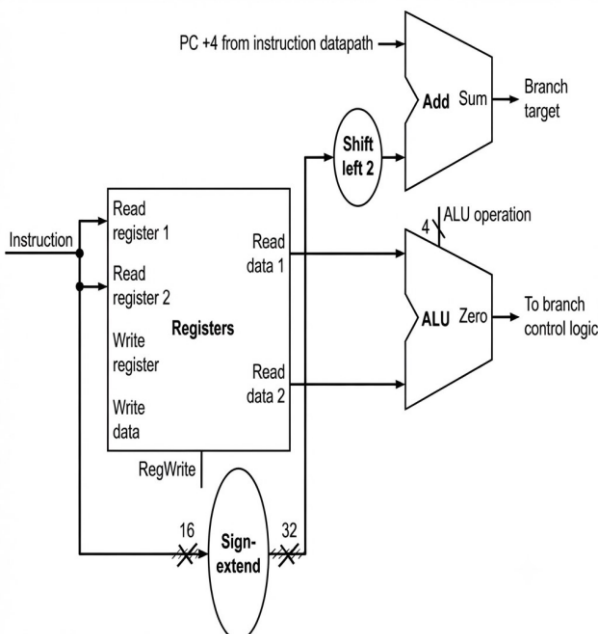
1- حساب عنوان هدف الفرع $(pc + 4) + (offset * 4)$

2- مقارنة بين محتويات $\$t1, \$t2$ نحتاج إلى :

- تمديد الإشارة - جامع - pc - ملف سجلات - Alu

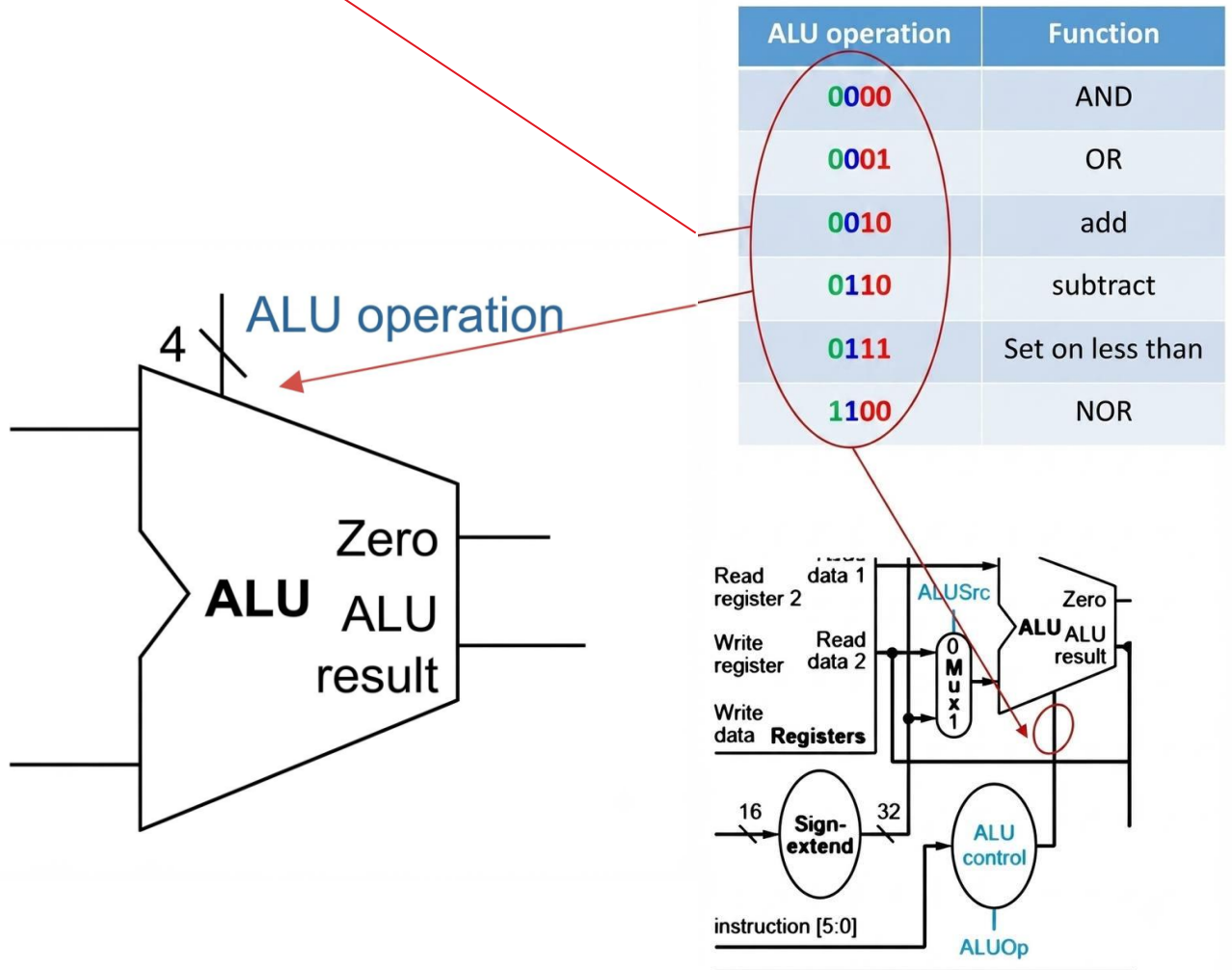
نأخذ الفرع If $\$t1 = \$t2$

لا نأخذ الفرع If $\$t1 \neq \$t2$



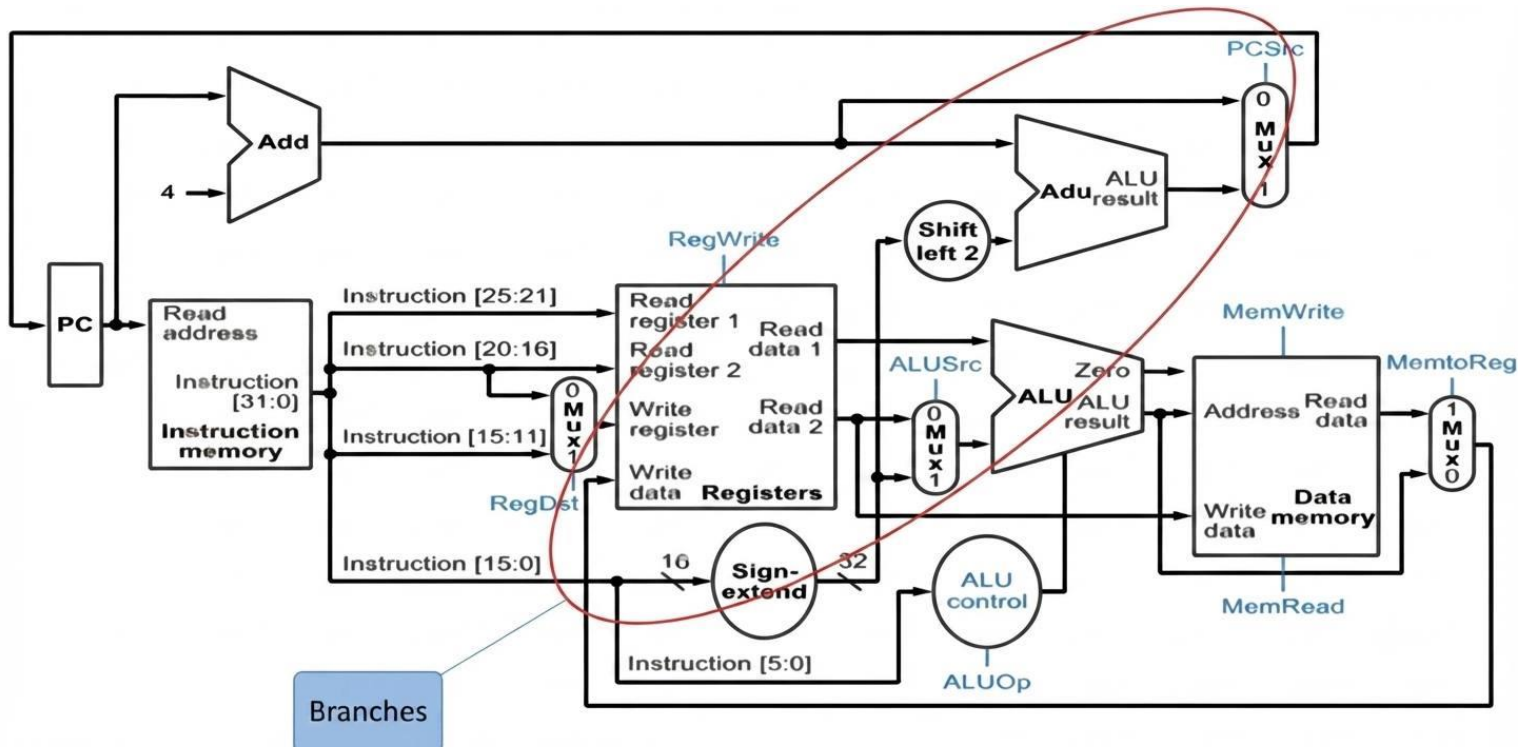
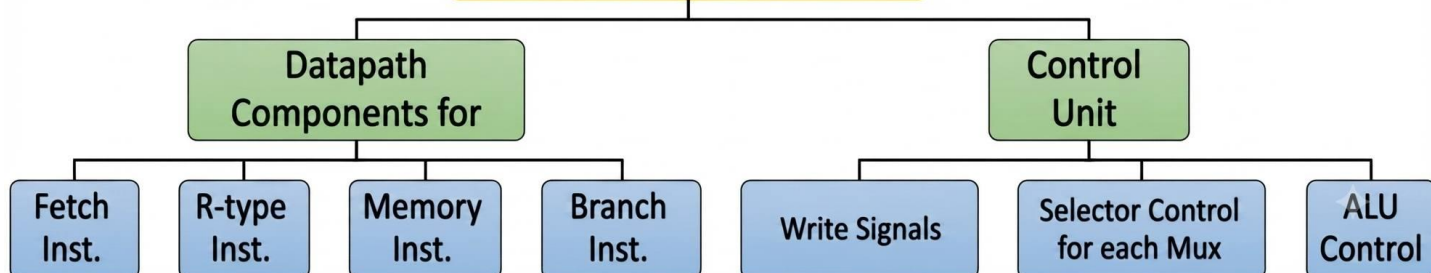
حتى نقوم بإنشاء مسار بيانات بسيط Simple Datapath

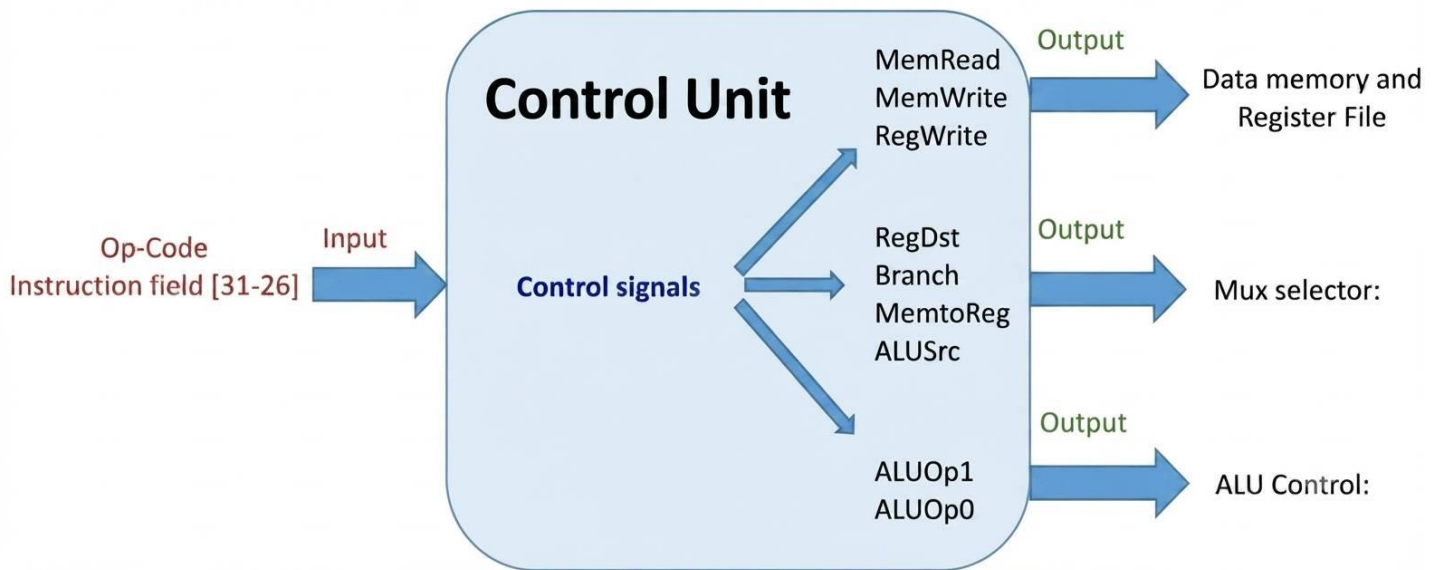
- اعتماد على صف التعليمات ستقوم ALU بوحدة من هذه الوظائف:
بالنسبة ل lw و sw تقوم ALU بحساب عنوان الذاكرة عن طريق الجمع
بالنسبة ل R-Type تقوم ALU بوحدة من add - sub - and - or (اعتماداً على حقل func)
- توليد خطوط ALU ذات 4 بت
حقل func ذو 6 بت
ذو 2 بت (ALU op) (مصمم في وحدة التحكم)
للتحميل والتخزين (تعليمات الذاكرة) أضف (00)
لتعليمات التفرع (beq) اطرح (01)
وحدة من العمليات في حقل func(10) تعليمات R-Type



Instruction opcode	ALUOp	Instruction operation	Funct field	Desired ALU action	ALU control input
LW	00	load word	XXXXXX	add	0010
SW	00	store word	XXXXXX	add	0010
Branch equal	01	branch equal	XXXXXX	subtract	0110
R-type	10	add	100000	add	0010
R-type	10	subtract	100010	subtract	0110
R-type	10	AND	100100	AND	0000
R-type	10	OR	100101	OR	0001
R-type	10	set on less than	101010	set on less than	0111

Simple datapath





في ختام هذه المادة، تذكروا دائماً أن العقل الذي بنيتموه هو الباب الحقيقي للمستقبل. لم يكن هدفنا فقط أن تنجحوا في الاختبار، بل أن تنجحوا في استخدام هذه المعرفة لتغيير واقعكم ... لأن اليأس جريمة والتسليم للواقع خيانة للفكر ... فخورون جداً كوننا جزءاً من رحلتكم، ونتمنى لكم التوفيق والتميز دائماً

